

UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

Subject:	EE-170L Computer Programming Lab
Lab No:	Lab 14
Lab Title:	Relative Grading System in C++
Semester:	Semester 2
Session:	Spring 2026

Submitted By:

Name: Shahzad Gul

Registration No: BF25nWELE0671

Date: May 2026

1. OBJECTIVE

To understand the UET Peshawar relative grading mechanism and implement it in C++. To learn normalization of raw marks using the formula: $\text{Normalized} = (\text{Actual} / \text{Highest}) \times 100$. To map normalized marks to letter grades across ten equal intervals from 50 to 100. To convert letter grades to GPA points on the standard 4.0 scale. To use arrays to store and process data for an entire class of students and display formatted results with class statistics.

2. THEORY

Relative grading (also called grading on a curve) assigns letter grades based on a student's performance relative to the rest of the class rather than against a fixed percentage scale. UET Peshawar uses relative grading for Type-A (theory) courses and absolute grading for Type-B (lab/practical) courses. Under the relative system, the highest scorer in the class is always normalized to 100, and all other marks are scaled proportionally.

The Normalization Formula is: $\text{Normalized Marks} = (\text{Actual Marks} / \text{Highest Marks in Class}) \times 100$. This guarantees that the top scorer always receives a normalized mark of 100, regardless of how difficult the exam was. All other students are scaled proportionally.

The range 50–100 is divided into ten equal intervals of 5 marks each. Students with normalized marks below 50 receive an F. The student who achieves the maximum threshold (100) receives an A+. Each letter grade corresponds to a fixed grade point on the 4.0 scale. A and A+ both carry 4.00 points; A+ is an honorary distinction for the class topper but does not raise the GPA above 4.00. The minimum passing grade is D (1.00).

3. LAB TASKS

3.1 Single-Student Grade Calculator

This minimal example demonstrates normalization and grade assignment for one student, illustrating the core mathematical steps before scaling to a full class.

Source Code:

```
#include <iostream>
#include <string>
using namespace std;

// =====
// Function: assignGrade
// =====
// Maps normalized marks to a letter grade
// Range 50-100 split into 10 equal intervals of 5
// Below 50 = F, Exactly 100 = A+ (top scorer)
string assignGrade(float nm)
{
    if (nm >= 100) return "A+"; // Maximum threshold: class topper
    if (nm >= 95)  return "A";  // 95-99.99
    if (nm >= 90)  return "A-"; // 90-94.99
```

```

        if (nm >= 85)    return "B+";    // 85-89.99
        if (nm >= 80)    return "B";     // 80-84.99
        if (nm >= 75)    return "B-";    // 75-79.99
        if (nm >= 70)    return "C+";    // 70-74.99
        if (nm >= 65)    return "C";     // 65-69.99
        if (nm >= 60)    return "C-";    // 60-64.99
        if (nm >= 55)    return "D+";    // 55-59.99
        if (nm >= 50)    return "D";     // 50-54.99: minimum passing grade
        return "F";      // Below 50: failed, must repeat course
    }

int main()
{
    // =====
    // Variable Declaration
    // =====
    float actual;    // Student's actual/raw marks
    float highest;   // Highest marks in the class

    // =====
    // Input Section
    // =====
    cout << "Enter student marks : ";
    cin >> actual;    // Read student's actual marks

    cout << "Enter highest marks : ";
    cin >> highest;   // Read highest marks in class

    // =====
    // Normalization Calculation
    // =====
    // Formula: Normalized = (Actual / Highest) * 100
    float nm = (actual / highest) * 100;

    // =====
    // Display Results
    // =====
    cout << "Normalized : " << nm << endl;
    cout << "Grade      : " << assignGrade(nm) << endl;

    return 0;
}

```

Expected Output:

```

Enter student marks : 74
Enter highest marks : 92
Normalized : 80.4348
Grade      : B

```

3.2 Class Grading with Arrays

This example extends the single-student approach to a full class using arrays and a loop to find the highest mark automatically before normalizing all students.

Source Code:

```
#include <iostream>
#include <string>
using namespace std;

// =====
// Function: assignGrade
// =====
// Maps normalized marks to a letter grade
string assignGrade(float nm)
{
    if (nm >= 100) return "A+";
    if (nm >= 95)  return "A";
    if (nm >= 90)  return "A-";
    if (nm >= 85)  return "B+";
    if (nm >= 80)  return "B";
    if (nm >= 75)  return "B-";
    if (nm >= 70)  return "C+";
    if (nm >= 65)  return "C";
    if (nm >= 60)  return "C-";
    if (nm >= 55)  return "D+";
    if (nm >= 50)  return "D";
    return "F";
}

int main()
{
    // =====
    // Variable and Array Declaration
    // =====
    int n;                // Number of students
    float marks[50];       // Array to store marks of up to 50 students
    float highest = 0;     // Variable to track highest mark in class

    // =====
    // Input Section
    // =====
    cout << "Enter number of students: ";
    cin >> n;

    // Loop to input marks and find highest mark simultaneously
    for (int i = 0; i < n; i++)
    {
```

```

        cout << "Marks[" << i + 1 << "]: ";
        cin >> marks[i];           // Read marks for each student

        // Update highest mark if current student scored higher
        if (marks[i] > highest)
            highest = marks[i];
    }

    // =====
    // Processing and Output Section
    // =====
    cout << "\nResult Table:" << endl;

    // Loop through each student to normalize and assign grades
    for (int i = 0; i < n; i++)
    {
        // Normalize marks using formula: (actual / highest) * 100
        float nm = (marks[i] / highest) * 100;

        cout << "Student " << i + 1
              << "   Norm = " << nm
              << "   Grade = " << assignGrade(nm) << endl;
    }

    return 0;
}

```

Expected Output:

```

Enter number of students: 3
Marks[1]: 90
Marks[2]: 75
Marks[3]: 45

Result Table:
Student 1   Norm = 100   Grade = A+
Student 2   Norm = 83.33   Grade = B
Student 3   Norm = 50    Grade = D

```

3.3 Full Relative Grading System (Complete Implementation)

Implement a complete relative grading system in C++ for a class of up to 100 students. The program must: accept student names and marks, find the highest mark automatically, normalize all marks, assign letter grades according to the UET Peshawar scheme, convert grades to GPA points, display a formatted result table, and show class statistics including pass/fail count, class average, and grade distribution.

Source Code:

```

// =====
// Lab 14 - Relative Grading System (UET Peshawar)
// Course : EE-170L Computer Programming Lab
// Name   : Muhammad Adil | Reg: BF25nWELE0699
// Date    : May 2026
// =====

#include <iostream>    // for cin, cout, endl
#include <iomanip>      // for setw(), setprecision(), fixed
#include <string>       // for string type and comparisons
using namespace std;

// =====
// Constant Declaration
// =====
// Maximum number of students the program can handle
const int MAX_STUDENTS = 100;

// =====
// Function Prototypes
// =====
float normalizeMarks(float actual, float highest);
string assignGrade(float normalized);
float gradeToGPA(string grade);
void printLine(char ch, int n);
void printTable(string names[], float actual[],
                float normalized[], string grades[],
                float gpa[], int n);
void printStats(float actual[], float normalized[],
                string grades[], float gpa[], int n);

// =====
// Main Function
// =====
int main()
{
    // Variable Declaration
    int n;                                // Number of students
    string names[MAX_STUDENTS];           // Array for student names
    float actual[MAX_STUDENTS];           // Array for raw marks
    float normalized[MAX_STUDENTS];       // Array for scaled marks (0-100)
    string grades[MAX_STUDENTS];          // Array for letter grades
    float gpa[MAX_STUDENTS];              // Array for GPA points
    float highest = 0;                    // Tracks highest mark in class

    // =====
    // Display Program Header
    // =====
    printLine('-', 60);

```

```

cout << "UET PESHAWAR - RELATIVE GRADING SYSTEM\n";
printLine('-', 60);

// =====
// Read Total Number of Students
// =====
cout << "Enter total number of students: ";
cin >> n;

// =====
// Input Loop: Collect Name and Marks for Each Student
// =====
for (int i = 0; i < n; i++)
{
    cout << "Student " << i + 1 << " Name : ";
    cin.ignore(); // Flush newline left by previous
cin >>
    getline(cin, names[i]); // getline() captures names with
spaces

    cout << "Student " << i + 1 << " Marks : ";
    cin >> actual[i];

    // Validate: marks must be between 0 and 100
    while (actual[i] < 0 || actual[i] > 100)
    {
        cout << "Invalid. Enter 0-100: ";
        cin >> actual[i];
    }

    // Update highest mark if this student scored higher
    if (actual[i] > highest)
        highest = actual[i];
}

// =====
// Processing Loop: Normalize, Assign Grade, Convert to GPA
// =====
for (int i = 0; i < n; i++)
{
    normalized[i] = normalizeMarks(actual[i], highest); // Scale marks
    grades[i] = assignGrade(normalized[i]); // Map to
letter
    gpa[i] = gradeToGPA(grades[i]); // Map to
points
}

// =====
// Display Formatted Results Table and Class Statistics

```

```

    // =====
    printTable(names, actual, normalized, grades, gpa, n);
    printStats(actual, normalized, grades, gpa, n);

    return 0;
}

// =====
// Function: normalizeMarks
// =====
// Formula: Normalized = (Actual / Highest) * 100
// Scales all marks so the top scorer always gets 100
float normalizeMarks(float actual, float highest)
{
    // Guard against division by zero
    if (highest == 0)
        return 0;

    return (actual / highest) * 100;
}

// =====
// Function: assignGrade
// =====
// Maps normalized marks to a letter grade
// Range 50-100 split into 10 equal intervals of 5
// Below 50 = F, Exactly 100 = A+ (top scorer)
string assignGrade(float nm)
{
    if (nm >= 100) return "A+";    // Maximum threshold: class topper
    if (nm >= 95)  return "A";     // 95-99.99: excellent
    if (nm >= 90)  return "A-";    // 90-94.99: excellent
    if (nm >= 85)  return "B+";    // 85-89.99: very good
    if (nm >= 80)  return "B";     // 80-84.99: above average
    if (nm >= 75)  return "B-";    // 75-79.99: above average
    if (nm >= 70)  return "C+";    // 70-74.99: above average (low)
    if (nm >= 65)  return "C";     // 65-69.99: average
    if (nm >= 60)  return "C-";    // 60-64.99: average
    if (nm >= 55)  return "D+";    // 55-59.99: average (low)
    if (nm >= 50)  return "D";     // 50-54.99: minimum passing grade
    return "F";                  // Below 50: failed, must repeat course
}

// =====
// Function: gradeToGPA
// =====
// Converts a letter grade string to a GPA point (4.0 scale)
// A and A+ both return 4.00; A+ is an honorary distinction only
float gradeToGPA(string g)

```



```

{
    if (g == "A+" || g == "A")    return 4.00f;    // Excellent
    if (g == "A-")                return 3.67f;    // Excellent
    if (g == "B+")                return 3.33f;    // Very Good
    if (g == "B")                 return 3.00f;    // Above Average
    if (g == "B-")                return 2.67f;    // Above Average
    if (g == "C+")                return 2.33f;    // Above Average (low)
    if (g == "C")                 return 2.00f;    // Average
    if (g == "C-")                return 1.67f;    // Average (low)
    if (g == "D+")                return 1.33f;    // Minimum Acceptable
    if (g == "D")                 return 1.00f;    // Minimum Acceptable
    return 0.00f;                  // F: Failed
}

// =====
// Function: printLine
// =====
// Prints n copies of character ch followed by a newline
// Used to draw separator lines in the output
void printLine(char ch, int n)
{
    for (int i = 0; i < n; i++)
        cout << ch;
    cout << "\n";
}

// =====
// Function: printTable
// =====
// Displays a formatted result table with all student data
// setw() pads each column to a fixed width for alignment
void printTable(string names[], float actual[],
                float normalized[], string grades[],
                float gpa[], int n)
{
    // Print column headers
    cout << "\n";
    printLine('=', 75);
    cout << left << setw(4) << "#"
        << setw(20) << "Name"
        << setw(10) << "Marks"
        << setw(14) << "Normalized"
        << setw(8) << "Grade"
        << setw(6) << "GPA" << "\n";
    printLine('-', 75);

    // Print one row per student
    for (int i = 0; i < n; i++)
    {
        cout << left << setw(4) << (i + 1)

```

```

        << setw(20) << names[i]
        << setw(10) << fixed << setprecision(2) << actual[i]
        << setw(14) << normalized[i]
        << setw(8) << grades[i]
        << setw(6) << gpa[i] << "\n";
    }
    printLine('=', 75);
}

// =====
// Function: printStats
// =====
// Calculates and displays class-wide statistics:
// total passed/failed, average raw marks, and average GPA
void printStats(float actual[], float normalized[],
               string grades[], float gpa[], int n)
{
    float sumM = 0, sumN = 0, sumG = 0; // Running totals
    int pass = 0, fail = 0;             // Pass/fail counters

    // Single pass: accumulate sums and count pass/fail
    for (int i = 0; i < n; i++)
    {
        sumM += actual[i];           // Total raw marks
        sumN += normalized[i];       // Total normalized marks
        sumG += gpa[i];              // Total GPA points

        if (grades[i] == "F")
            fail++;                  // F = failed
        else
            pass++;                  // Any other grade = passed
    }

    // Display computed statistics
    cout << "\n===== CLASS STATISTICS =====" << endl;
    cout << "Passed      : " << pass << endl;
    cout << "Failed      : " << fail << endl;
    cout << "Class Avg   : " << sumM / n << " marks" << endl;
    cout << "Avg GPA     : " << sumG / n << endl;
    cout << "===== " << endl;
}

```

Expected Output:

```

=====
UET PESHAWAR - RELATIVE GRADING SYSTEM
=====
Enter total number of students: 5
Student 1 Name : Ali Khan

```

```

Student 1 Marks : 78
Student 2 Name : Ahmed Raza
Student 2 Marks : 92
Student 3 Name : Fatima Noor
Student 3 Marks : 55
Student 4 Name : Bilal Mehmood
Student 4 Marks : 45
Student 5 Name : Sara Ahmad
Student 5 Marks : 88

```

```

=====
==
#      Name                Marks      Normalized      Grade      GPA
-----
--
1      Ali Khan             78.00      84.78           B          3.00
2      Ahmed Raza           92.00      100.00          A+         4.00
3      Fatima Noor          55.00      59.78           D+         1.33
4      Bilal Mehmood        45.00      48.91           F          0.00
5      Sara Ahmad           88.00      95.65           A          4.00
=====
==

===== CLASS STATISTICS =====
Passed      : 4
Failed      : 1
Class Avg   : 71.60 marks
Avg GPA     : 2.47
=====

```

Explanation:

The program first collects all marks and tracks the highest score in one pass through the loop. Normalization is applied to every student using the ratio $(\text{actual} / \text{highest}) \times 100$, so the top scorer always receives 100. The `assignGrade()` function maps the normalized mark to a letter grade using cascading if-else comparisons in descending order. The `gradeToGPA()` function uses string comparison to return the corresponding 4.0-scale point value. Input validation inside the loop rejects marks outside the 0-100 range before they affect the highest-mark tracker.

4. RELATIVE VS ABSOLUTE GRADING

Key differences between relative and absolute grading systems:

Feature	Absolute Grading	Relative Grading
Grade boundary	Fixed (e.g. 85% = A)	Varies with class performance

Top scorer	May or may not get A	Always normalized to 100 (A+)
Exam difficulty	Directly impacts grades	Cancelled out by normalization
Used for	Type-B (Lab) courses	Type-A (Theory) courses
F grade trigger	Below fixed threshold	Normalized marks < 50

5. CONCLUSION

This lab implemented the complete UET Peshawar relative grading system in C++. Key takeaways: (1) Normalization Formula: Dividing each student's marks by the highest mark and multiplying by 100 ensures the top scorer always maps to 100, regardless of exam difficulty. (2) Grade Assignment: The 50-100 normalized range is split into ten 5-mark intervals, each corresponding to one of ten letter grades (D through A+). Below 50 is F; at 100 is A+. (3) GPA Mapping: Each letter grade carries a fixed point value on the 4.0 scale. A and A+ both yield 4.00 — A+ is an honorary distinction only. (4) Arrays and Loops: Storing student data in parallel arrays and processing them in loops is the standard C++ technique for batch data operations. (5) Input Validation: Rejecting out-of-range marks before they enter the calculation prevents corruption of the highest-mark tracker and ensures correct normalization for all students.

Relative grading is fundamentally a statistical tool: it rewards performance above the class average and penalizes performance below it. Mastering its implementation in C++ reinforces array manipulation, function design, and formatted output — skills that underpin every larger software project.